Final Report: Neural Network for Handwritten Digits

Daniel Akl, Jonathon Tucker, Kamrin Bains

High Level Framework:

Our project sought to train a Neural Network to classify handwritten digits. We trained on the MNIST (Modified National Institute of Standards and Technology database) dataset which consists of 60,000 28x28 images. Our top goal of the project was to train a model that would produce the highest accuracy when tested against our class dataset which consisted of 290 images. This report details our findings throughout each iteration of the model.

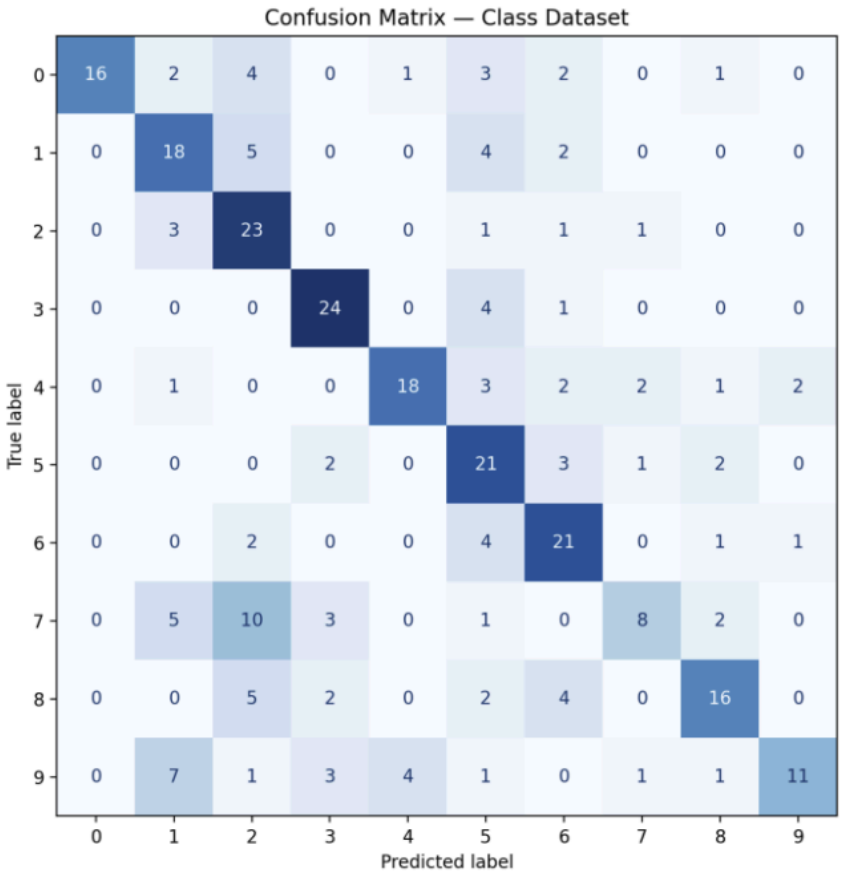
First Model Architecture:

Our first training runs consisted of three hidden layers which was by far our best results.

Results:

```
Accuracy on class dataset: 60.69% (176/290)

Per-digit accuracy:
  Digit 0: 16/29 (55.2%)
  Digit 1: 18/29 (62.1%)
  Digit 2: 23/29 (79.3%)
  Digit 3: 24/29 (82.8%)
  Digit 4: 18/29 (62.1%)
  Digit 5: 21/29 (72.4%)
  Digit 6: 21/29 (72.4%)
  Digit 7: 8/29 (27.6%)
  Digit 8: 16/29 (55.2%)
  Digit 9: 11/29 (37.9%)
```



It was decided to rework the code from this point after listening to one group's findings. Claiming they received amazing results by only using two hidden layers. The two-layer architecture was implemented yet we had a lower accuracy than using three. This is what we would expect as more layers offers more opportunities for the model to refine its decision making. I unfortunately didn’t take screenshots of the code during this time - only the outputs of the changes I made; Looking back I should have taken more measures to keep each iteration isolated to fully be able to examine it more in depth.

Final Model Architecture:

- Input layer: 784 neurons (28×28 images flattened)

- Hidden Layer 1: 256 neurons (ReLU)
- Hidden Layer 2: 128 neurons (ReLU)
- Output Layer: 10 neurons (LogSoftmax)

Implementation:

```python
# Training
loss_function = nn.NLLLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)


epochs = 15
for epoch in range(epochs):
    for images, labels in train_loader:
        optimizer.zero_grad()
        images = images.view(images.shape[0], -1)
        output = model(images)
        loss = loss_function(output, labels)
        loss.backward()
        optimizer.step()
    print(f'Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.4f}')
```

Training the Neural Net

Key Hyperparameters:

Learning Rate: 0.001 (optimal)

Epochs: 15 → later increased to 30

Optimizer: Adam

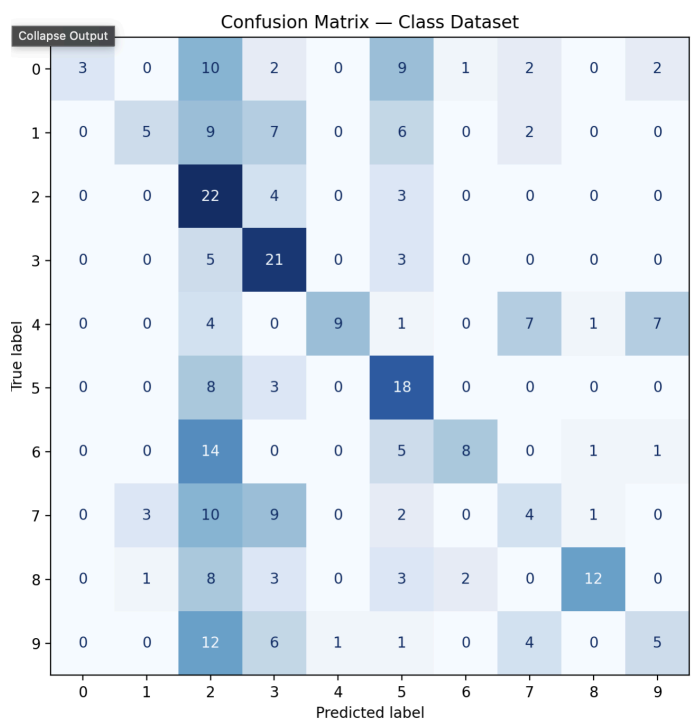
Loss Function: Negative Log Likelihood (NLLLoss)

- Learning rate tuning produced the largest performance gain (~14%).

Results:

Training Run 1:

- 0.01 Learning Rate
- Epoch Size 15
- Accuracy on Class Dataset



was 36.9%

- Error Rate: ~64%
- Predicts 2 and 3 fairly accurate, struggles with the rest

Notes:

- Model performance was unstable
- The confusion matrix showed a heavy bias toward predicting the digit 2
- Digits 2 and 3 appeared accurate only because the network over predicted similar shapes
- Generalization on real handwritten class data was poor

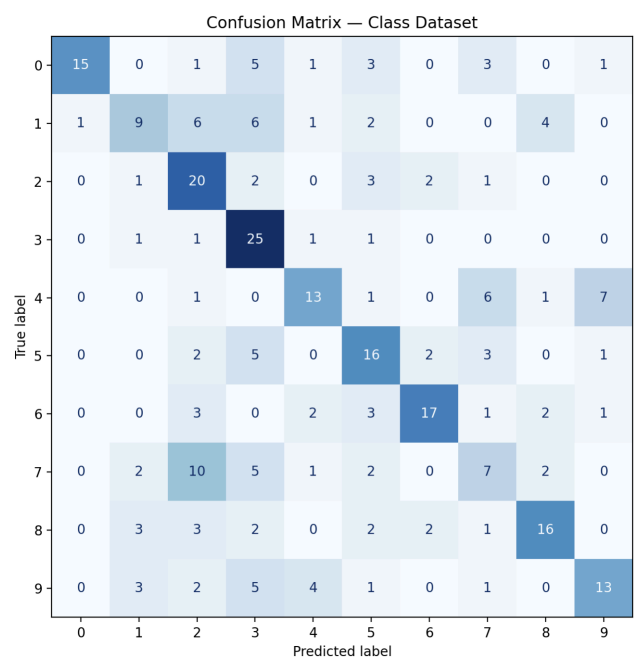
A learning rate of 0.01 was too high, causing inconsistent convergence and unreliable predictions.

Training Run 2:

- 0.001 Learning Rate
- Epoch Size 15
- Error Rate: 48%
- Accuracy Increased to 52.07%, around 14% increase

```
Accuracy on class dataset: 52.07% (151/290)

Per-digit accuracy:
  Digit 0: 15/29 (51.7%)
  Digit 1: 9/29 (31.0%)
  Digit 2: 20/29 (69.0%)
  Digit 3: 25/29 (86.2%)
  Digit 4: 13/29 (44.8%)
  Digit 5: 16/29 (55.2%)
  Digit 6: 17/29 (58.6%)
  Digit 7: 7/29 (24.1%)
  Digit 8: 16/29 (55.2%)
  Digit 9: 13/29 (44.8%)
```



Notes:

- Reducing the learning rate significantly improved accuracy
- The confusion matrix showed a much stronger diagonal → better results
- The network still struggled with: 1 & 7, 4 & 9

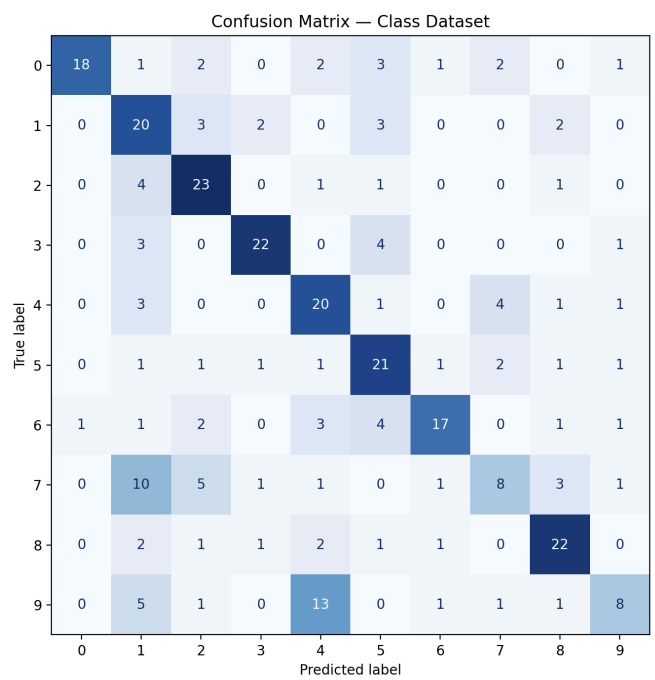
Changing the learning rate from 0.01 → 0.001 produced the largest improvement, increasing accuracy by nearly 14%

Training Run 3:

- 0.001 Learning Rate
- Epoch Size 30
- Accuracy Increased to 59.67%

```
Accuracy on class dataset: 59.67% (179/300)

Per-digit accuracy:
  Digit 0: 18/30 (60.0%)
  Digit 1: 20/30 (66.7%)
  Digit 2: 23/30 (76.7%)
  Digit 3: 22/30 (73.3%)
  Digit 4: 20/30 (66.7%)
  Digit 5: 21/30 (70.0%)
  Digit 6: 17/30 (56.7%)
  Digit 7: 8/30 (26.7%)
  Digit 8: 22/30 (73.3%)
  Digit 9: 8/30 (26.7%)
```



Notes:

- This was the best performing model
- Increasing epochs improved convergence and feature learning
- Confusion matrix showed the strongest diagonal → best results

- Remaining weaknesses:
    - Misclassification between 7 & 1, 4 & 9 (visually similar digits)

Longer training improved model generalization while maintaining strong MNIST performance.

Conclusion

The neural network learned digit classification with:

- High performance on MNIST (>96%)
- Moderate performance on real-world class data (~60%)

Key Takeaways:

- Hyperparameter tuning (especially learning rate) is crucial
- More epochs improve accuracy but with diminishing returns
- Real world data (that isn't optimized) introduces chaos and reduces performance